

ORCS E4200 Project Proposal: Baby Blackjack Bandit

Donovan Barcelona and Peter Wu

Motivation

Imagine placing a newborn child at a blackjack table with a decent bankroll. The baby does not understand numbers, but it does love money. How long would it take for your baby to achieve decent returns, or even become a money printing machine?

In this project, we aim to create a winning baby by exploring different constraints within the blackjack problem. Some questions we hope to answer include:

- How strong are the baby's observational skills? Can it distinguish suits, and how many cards can it remember?
- Should the baby control its own bet size? Or would two "twin babies," one managing betting and one playing hands, perform better?
- What defines success for the baby—winning a hand or surviving a full shoe?
- How "intelligent" does the baby need to be? What model architectures can yield strong blackjack performance?

Approach & Related Work

Games, and especially gambling games, have long been used to test Reinforcement Learning (RL) frameworks due to their well-defined rules and easily simulated environments. From Atari¹ to poker², gameplay has long illustrated RL's ability to learn strategy from repeated iterations.

We are confident that our "baby" can approximate an optimal blackjack strategy. Despite the game's large state space, the action space is relatively compact, and an analytical "optimal" strategy for blackjack already exists as demonstrated by Kevin Spacey in 21. This makes it a useful environment for testing RL convergence and efficiency under various levels of domain knowledge and state observability.

In particular, we are interested in studying not only our baby's ability to converge to an optimal policy, but also how human knowledge (e.g., reward shaping, card counting, or hand heuristics) influences convergence speed and stability.

Workplan

1. **Implement Blackjack Environment** (in python) – Defining the deck, hand evaluation, betting rules, and game termination.
2. **Formulate as an RL Problem** (using *Gymnasium* framework), defining:

¹ Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. *Playing Atari with Deep Reinforcement Learning*. arXiv preprint [arXiv:1312.5602](https://arxiv.org/abs/1312.5602) (2013). <https://doi.org/10.48550/arXiv.1312.5602>

² Noam Brown and Tuomas Sandholm. *Superhuman AI for Multiplayer Poker*. *Science*, **365** (6456), 885–890 (2019). <https://doi.org/10.1126/science.aay2400>

- a. **State:** current_balance, exhausted_cards, current_hand, dealers_hand
 - b. **Actions:** bet_size, hit/stay
 - c. **Reward:** balance at termination
 - d. **Time horizon:** until ruin (balance = 0) or exhaustion of all cards in a deck
 - e. **Transition probability:** probabilistic card draws
3. Testing Three Settings (of increasing complexity)
 - a. **Simple blackjack.** The Baby plays individual hands, with a reshuffled deck each episode. Reward = 1 for a win, 0 for a loss.
 - b. **No-shuffle blackjack.** The Baby plays blackjack but the deck is not reshuffled between hands. Reward = number of wins before exhausting the deck.
 - c. **Blackjack with betting.** In addition to not reshuffling the deck, the baby decides how much to bet before playing a hand. Reward = final balance after the deck is exhausted.

Anticipated Challenges

The main difficulty is ensuring efficient and stable convergence. The state space is very large since the baby is “counting cards”, so efficient convergence will rely on how much we tailor the reward function & balance exploration/exploitation. Potential approaches include:

- Rewarding hands that are as close as possible to 21 and penalizing “busts” (going over 21)
- Using a decaying epsilon-greedy exploration
- Experimenting with different levels of state observability (e.g. allowing it to track suits as a red-herring versus just keeping a running tally of how many of each card has been dealt).

Another challenge is handling sequential betting (setting c). Unlike hit/stay actions, betting occurs once per hand rather than every timestep. We will need to design a way for the environment to “lock in” bet size until each hand concludes. This should be possible considering the existence of agents that handle gambling (e.g. poker bots), but requires some more research.

Ideal Deliverables

By the end of the project we hope to have (1) trained an RL agent that can play blackjack optimally, with adaptive bet-sizing, and (2) visualized how different information settings lead to different rates of convergence for our problem.

In addition to our summary report, we would love to make a visualization of our blackjack baby playing hands, showing its transformation over the training period to becoming a money printing machine.